Build Task — HTTP Client Wrapper — Hints & Solution

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint builds toward the full solution.

- Hint 1 — What you're building, and the exact pieces
- Hint 2 — The plan, and almost the whole thing
- Solution

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Hint 1 — What you're building, and the exact pieces

You're combining everything from this document's exercises into one function: `request_with_retry(method: str, url: str, **kwargs) -> dict`.

The core design decision is a **classification step**: every failure needs to be sorted into "transient" (worth retrying: timeout, 5xx, 429) or "permanent" (not worth retrying: any other 4xx). This classification should probably be its own small piece of logic, separate from the retry loop itself, so you can test it on its own.

The retry loop then wraps a single attempt, using your `compute_backoff_delay(attempt)` helper from the timeout/retry exercise, and stops either on success or once a permanent failure is classified (fail immediately, no retry) or the attempt limit is hit (raise `TransientHTTPError`).

Here's what to actually go look at:

- **`requests.request(method, url, timeout=..., **kwargs)`** — the generic entry point `requests.get` / `.post` /etc. all call underneath; taking `method` as a string parameter lets your one function handle every HTTP verb.
- **Classifying by status code** — `response.status_code == 429` is transient. `response.status_code // 100 == 5` is transient. `response.status_code // 100 == 4` (and not 429) is permanent. Anything else (2xx, 3xx) is success.

- **`requests.exceptions.Timeout`** and **`requests.exceptions.ConnectionError`** — both count as transient failures, same bucket as a 5xx.
- **`compute_backoff_delay(attempt) + random.uniform(0, jitter_max)`** — jitter is just adding a small random number on top of your existing exponential formula.
- **Attaching the response body to `PermanentHTTPError`** — store it as an attribute (`self.body = response.text`) in `__init__`, so a caller catching the error can actually see what the server said was wrong.

Sketch the function's overall shape — the loop, the classification call, and the two error types it can raise — before checking Hint 2.

Think past "classify the failure and retry it correctly." Ask: **this function is supposed to be testable without real network calls where possible, per the Build Task's own constraint — which parts of it can actually be tested that way, and how did the earlier exercises already show you?**

The answer isn't a new trick — it's the same move every exercise in this document already made: pull the *decision* logic out into its own small, pure function, and test that directly with plain values. `is_transient_status(status_code)` above is one such function. `compute_backoff_delay(attempt)` from the timeout/retry exercise is another. The rate-limit exercise's `decide_wait_seconds(headers, attempt)` is a third — it takes a plain `headers` dict, not a real response object, so it's testable the exact same way. The retry loop itself — the part that actually calls `requests.request` — gets exercised with a real call, the same way the timeout/retry exercise's `get_with_retry()` was: no fake server, nothing new to learn.

There's a second question worth asking too: should every call to `request_with_retry()` open a brand-new connection, or should repeated calls to the same host reuse one, the way the session-reuse exercise showed? A module-level `requests.Session()`, reused across calls, is both faster (connection pooling) and the natural place to set default headers once (like a User-Agent identifying your client) instead of on every call.

The extra pieces:

- A module-level `requests.Session()` in `http_client.py`, called as `_session.request(...)` inside the loop instead of the bare `requests.request`.
- `decide_wait_seconds(headers, attempt)` from the rate-limit exercise, called inside the retry loop before falling back to `compute_backoff_delay(attempt)` — a 429 should respect `Retry-After` if the server sent one, the same as any other real client.

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and almost the whole thing

`exceptions.py`:

```
class MissingConfigError is an Exception    (kept from Doc01)
class TransientHTTPError is an Exception
class PermanentHTTPError is an Exception, holds status_code and body
```

`http_client.py`:

```
function compute_backoff_delay(attempt) -> float:
    base = 2 ** attempt
    jitter = a random float between 0 and 1
    return base + jitter

function is_transient_status(status_code) -> bool:
    return status_code == 429 or status_code // 100 == 5

function request_with_retry(method, url, max_attempts=5, **kwargs) -> dict:
    for attempt in range(max_attempts):
        logger.debug(f"attempt {attempt}: {method} {url}")
        try:
            response = requests.request(method, url, timeout=(3, 10), **kwargs)
        except (Timeout, ConnectionError) as e:
            if attempt == max_attempts - 1:
                logger.error(f"gave up after {max_attempts} attempts: {e}")
                raise TransientHTTPError(str(e)) from e
            sleep(compute_backoff_delay(attempt))
            continue

        if response.status_code < 300:
            try:
                return response.json()
            except json.JSONDecodeError as e:
                raise TransientHTTPError(f"bad JSON in response: {e}") from e

        if is_transient_status(response.status_code):
            if attempt == max_attempts - 1:
                logger.error(
                    f"gave up after {max_attempts} attempts, "
                    f"last status {response.status_code}"
                )
                raise TransientHTTPError(
                    f"status {response.status_code} "
                    f"after {max_attempts} attempts"
                )
            sleep(compute_backoff_delay(attempt))
            continue
```

```
        # any other 4xx: permanent, no retry
        logger.error(f"permanent failure: status {response.status_code}")
        raise PermanentHTTPError(response.status_code, response.text)
```

The key structural idea: the loop has exactly two ways out — a successful return, or a raised error — and exactly one way to keep going — `continue` after sleeping. Nothing falls through silently. Write out the full, typed version yourself, then go one step further — Approach 2, which folds in the session-reuse and rate-limit exercises.

Solution

`exceptions.py`

```
# kept from Doc01, unchanged — config.py imports it from this file
# Why: gives load_config() its own error type, so calling code can catch a
# missing
# setting specifically, instead of catching every possible Exception blindly.
class MissingConfigError(Exception):
    """Raised when a required setting is missing from the environment."""
    pass

class TransientHTTPError(Exception):
    """Raised when a retryable failure (timeout, 5xx, 429) exhausts all
    attempts."""
    pass

class PermanentHTTPError(Exception):
    """Raised immediately for a non-retryable 4xx, with the response body
    attached."""

    def __init__(self, status_code: int, body: str) -> None:
        self.status_code = status_code
        self.body = body
        super().__init__(f"HTTP {status_code}: {body[:200]}")
```

`http_client.py`

```
import json
import random
import time
from typing import Any

import requests
```

```
from exceptions import PermanentHTTPError, TransientHTTPError
from logging_setup import get_logger

logger = get_logger(__name__)

DEFAULT_TIMEOUT = (3, 10)

def compute_backoff_delay(attempt: int) -> float:
    return (2 ** attempt) + random.uniform(0, 1)

def request_with_retry(
    method: str,
    url: str,
    max_attempts: int = 5,
    **kwargs: Any,
) -> dict:
    for attempt in range(max_attempts):
        logger.debug(f"attempt {attempt}: {method} {url}")
        try:
            response = requests.request(
                method, url, timeout=DEFAULT_TIMEOUT, **kwargs
            )
        except (
            requests.exceptions.Timeout,
            requests.exceptions.ConnectionError,
        ) as e:
            if attempt == max_attempts - 1:
                logger.error(f"gave up after {max_attempts} attempts: {e}")
                raise TransientHTTPError(str(e)) from e
            time.sleep(compute_backoff_delay(attempt))
            continue

        if response.status_code < 300:
            try:
                return response.json()
            except json.JSONDecodeError as e:
                raise TransientHTTPError(f"bad JSON in response: {e}") from e

        if response.status_code == 429 or response.status_code // 100 == 5:
            if attempt == max_attempts - 1:
                logger.error(f"gave up, last status {response.status_code}")
                raise TransientHTTPError(
                    f"status {response.status_code} "
                    f"after {max_attempts} attempts"
                )
            time.sleep(compute_backoff_delay(attempt))
```

```
            continue

        logger.error(f"permanent failure: status {response.status_code}")
        raise PermanentHTTPError(response.status_code, response.text)

if __name__ == "__main__":
    print(request_with_retry("GET", "https://api.github.com"))
```

Why this approach: everything lives in one place, which is easy to read start to finish for a project this size.